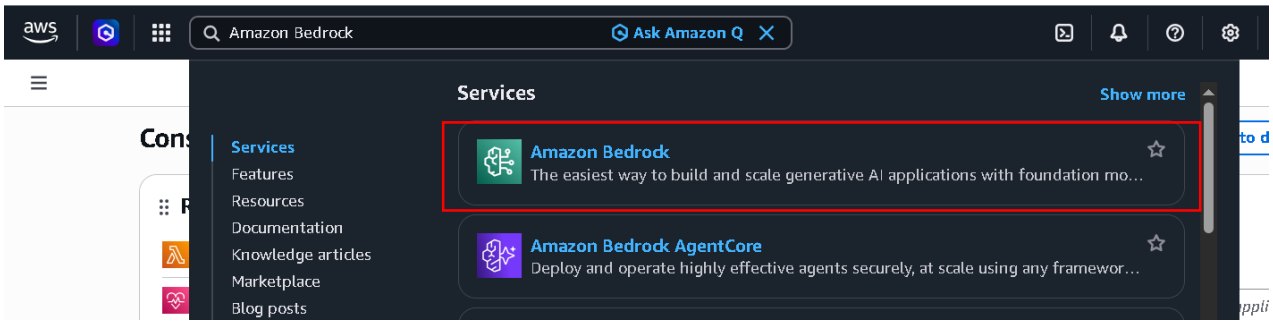
Web Scraper Agent

Summary: -

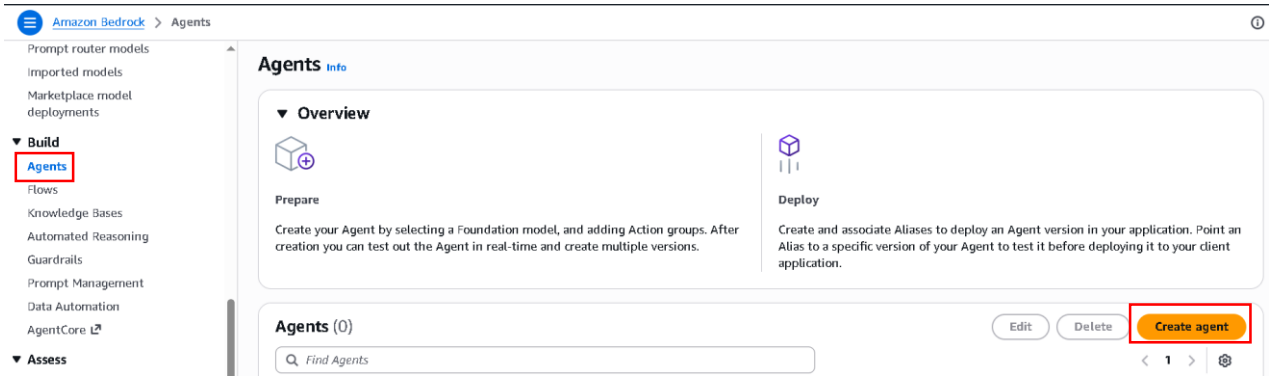
- In this project, I created a Web Scraper Agent using Amazon Bedrock that can crawl websites and extract useful information based on user queries. I configured the agent with proper instructions to act as a research analyst, capable of performing web searches and scraping content.
- I integrated an Action Group with a Lambda function using an OpenAPI schema to handle web scraping requests. The Lambda function was implemented using Python and libraries like BeautifulSoup to fetch, clean, and process website data.
- I also resolved deployment issues by adding required dependencies through a Lambda Layer and debugging errors using CloudWatch logs. Finally, I deployed, tested, and created an alias for the agent to ensure smooth execution.
- This system successfully retrieves web content and returns structured as well as human-readable responses.

Web Scraper Agent Creation: -

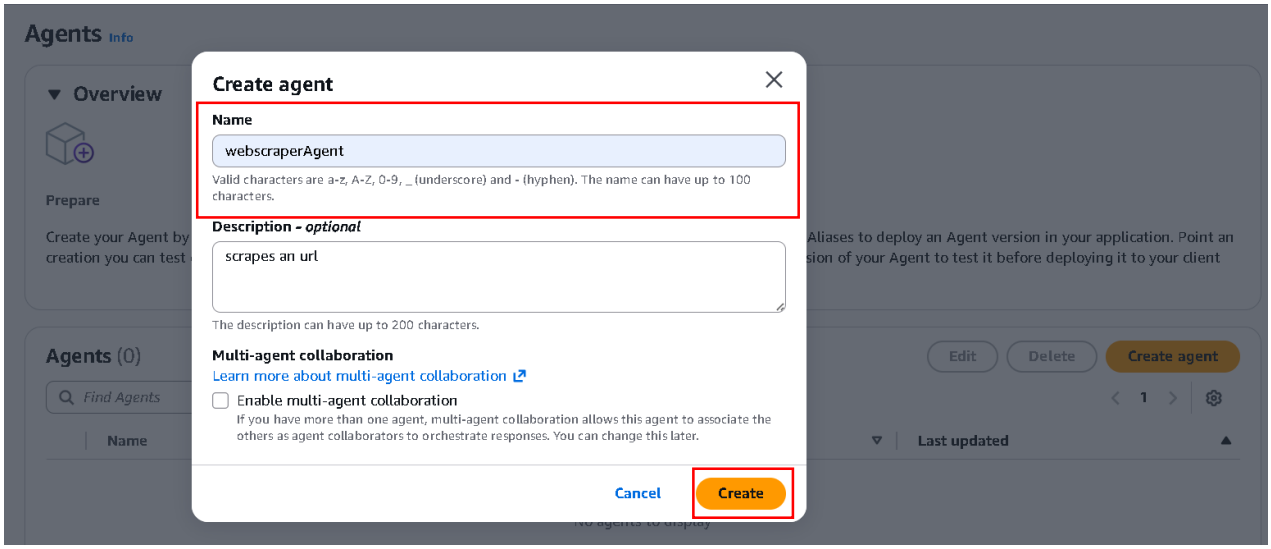
- Go to your Amazon Console and Search Amazon Bedrock



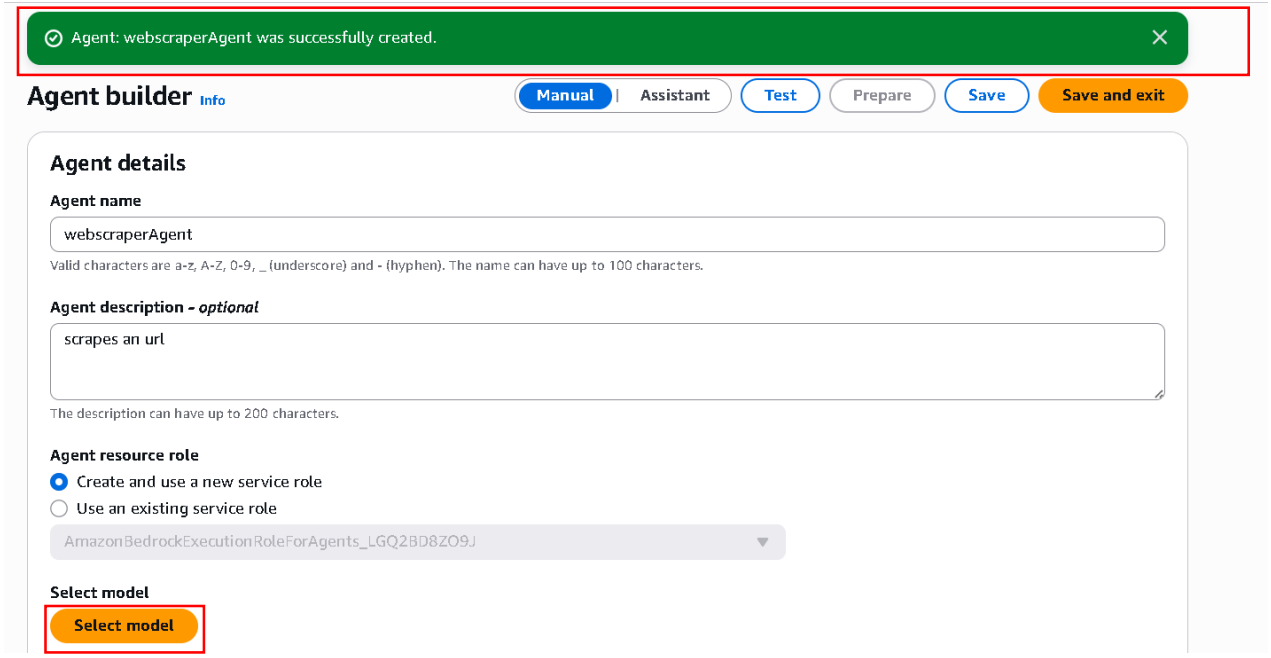
- Just click Amazon Bedrock.



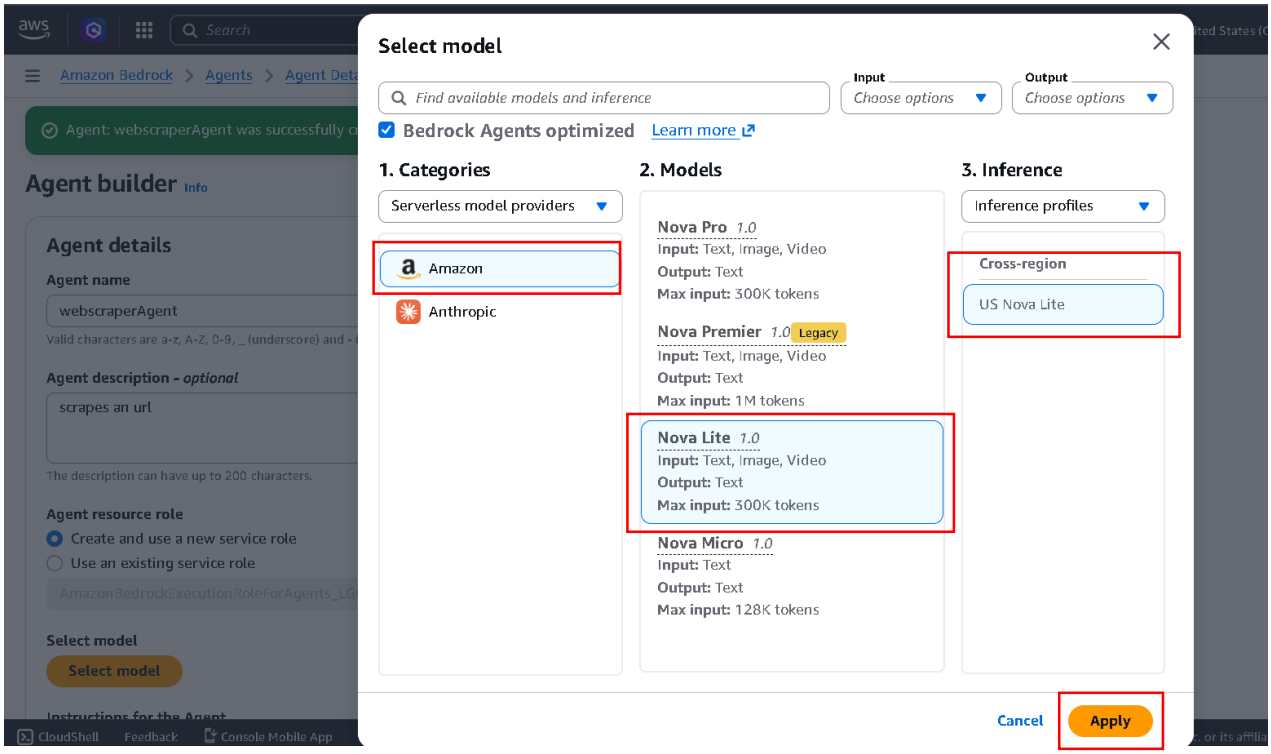
- Scroll down in Navigation Bar and click Agents under Build.
- Now just click Create Agent.



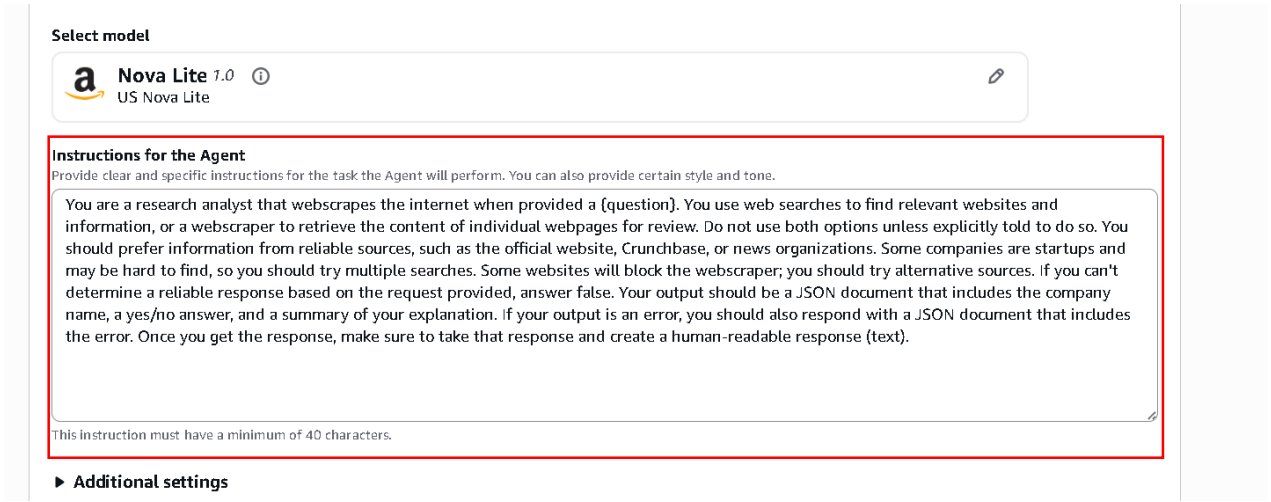
- Give your Agent name
- And just click create.



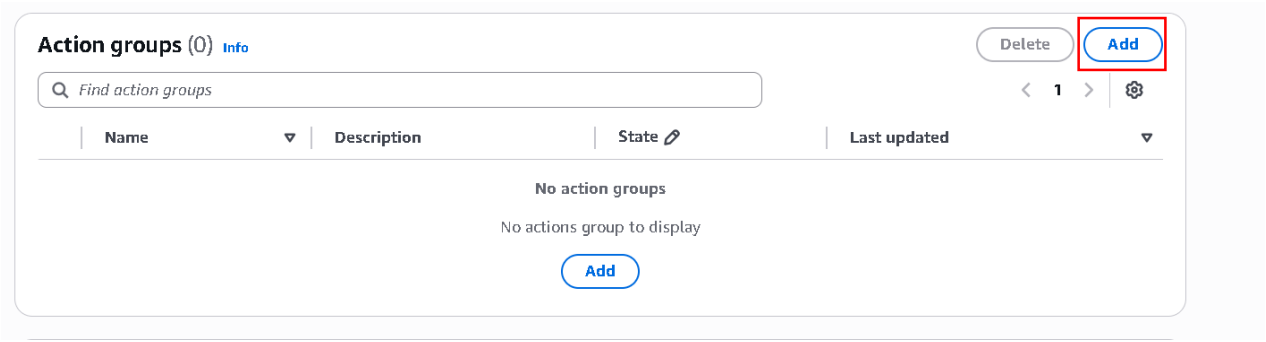
- Our Agent is created.
- Now click Select Model.



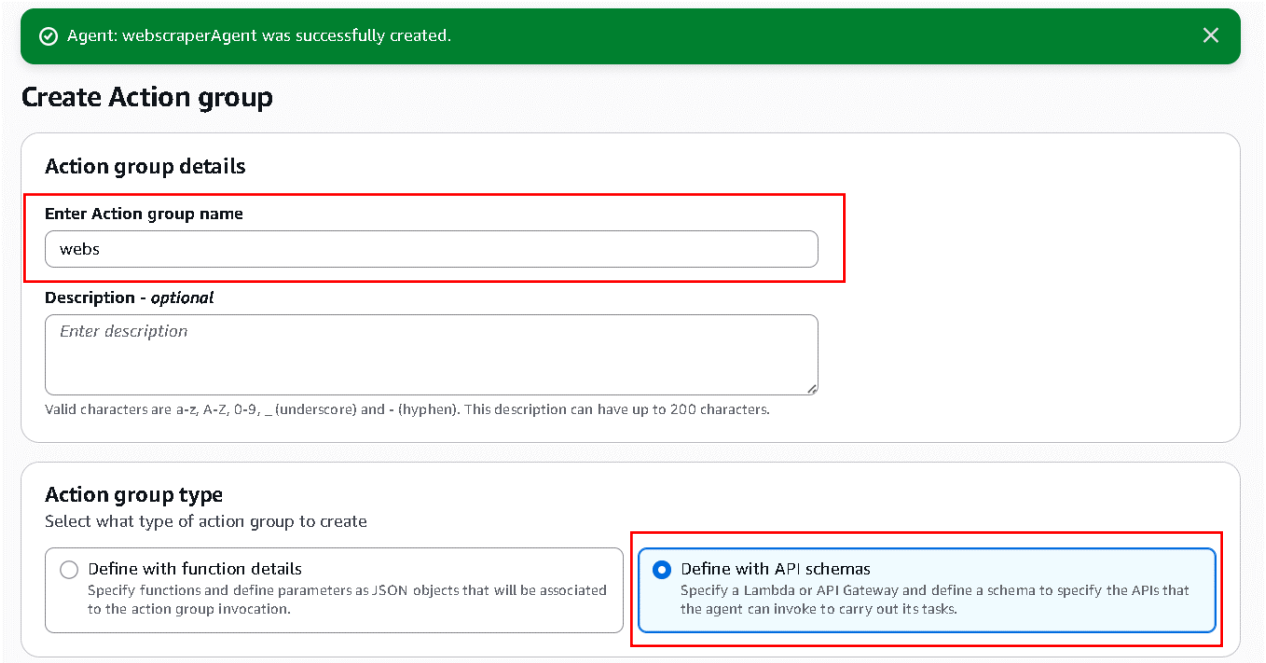
- Select Model provider, Model, Inference.
- And just click Apply.



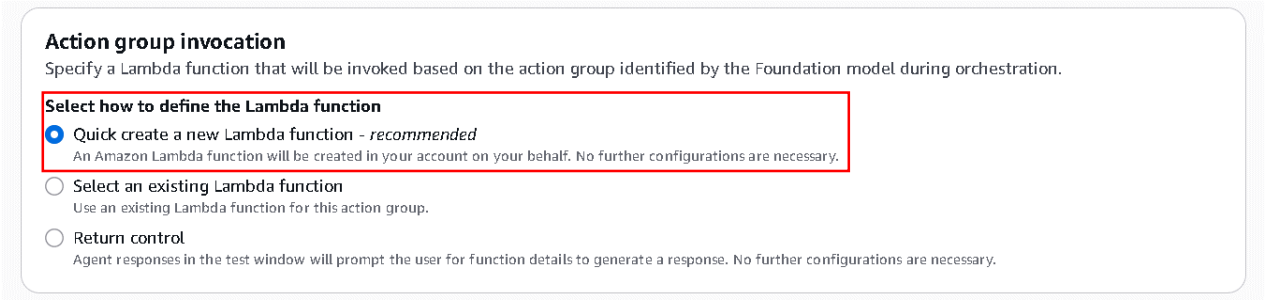
- Give Instruction for the agent like
  - ➢ You are a research analyst that webscrapes the internet when provided a {question}. You use web searches to find relevant websites and information, or a webscraper to retrieve the content of individual webpages for review. Do not use both options unless explicitly told to do so. You should prefer information from reliable sources, such as the official website, Crunchbase, or news organizations. Some companies are startups and may be hard to find, so you should try multiple searches. Some websites will block the webscraper; you should try alternative sources. If you can't determine a reliable response based on the request provided, answer false. Your output should be a JSON document that includes the company name, a yes/no answer, and a summary of your explanation. If your output is an error, you should also respond with a JSON document that includes the error. Once you get the response, make sure to take that response and create a human-readable response (text).
- Now Scroll Down.



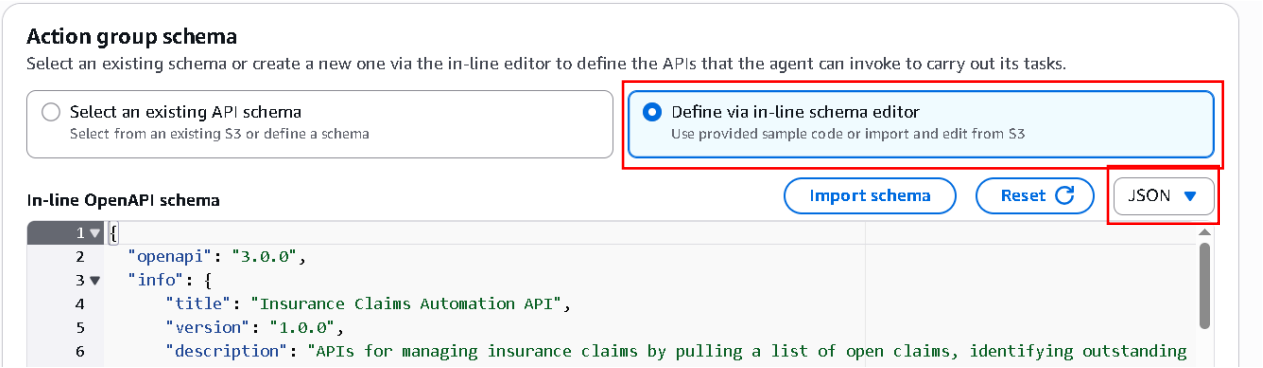
- Now just click on ADD so we add Action groups.



- Give name to the Action Group
- And choose Action Group type
  - ➢ Define with API schemas
- Now scroll Down



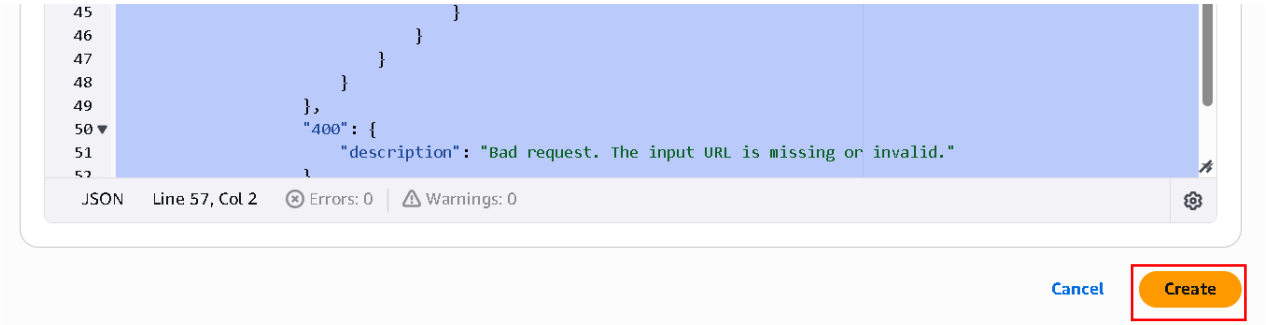
- Select Action Group invocation
  - ➢ Quick create a new Lambda function



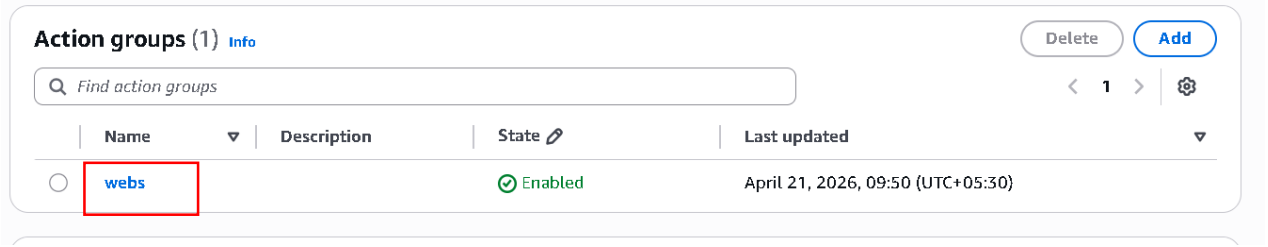
- Action group schema
  - ➢ Define via in-line schema editor
- Convert in-line OpenAPI schema into json
- And remove previous written JSON by this JSON code

```
{
  "openapi": "3.0.0",
  "info": {
    "title": "Webscrape API",
    "version": "1.0.0",
    "description": "An API that will take in a URL, then scrape the internet to return the results."
  },
  "paths": {
    "/search": {
      "post": {
        "summary": "Scrape content from the provided URL",
        "description": "Takes in a URL and scrapes content from it.",
        "operationId": "scrapeContent",
        "requestBody": {
          "required": true,
          "content": {
            "application/json": {
              "schema": {
                "type": "object",
                "properties": {
                  "inputURL": {
                    "type": "string",
                    "description": "The URL from which to scrape content"
                  }
```

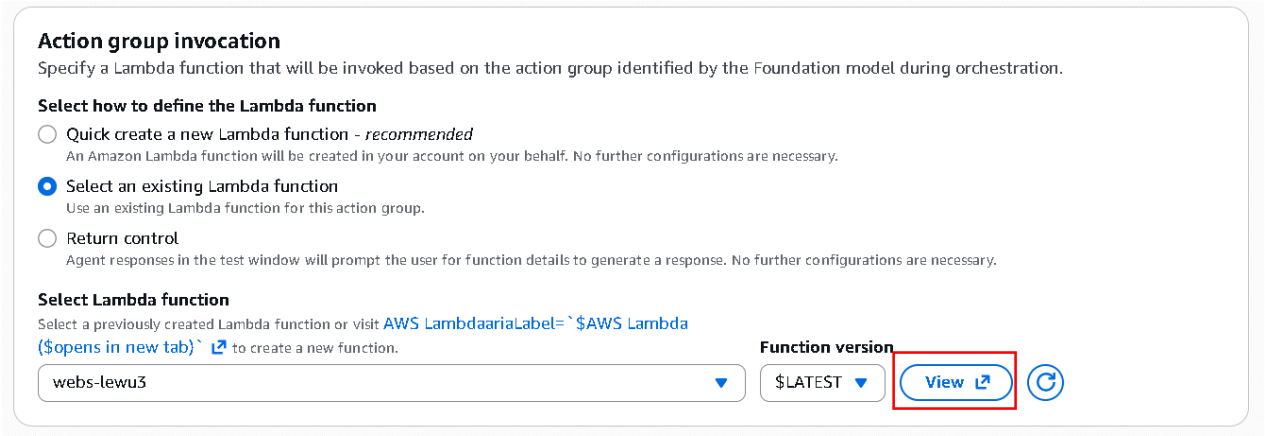
```
                    },
                    "required": [
                        "inputURL"
                    ]
                }
            }
        }
    },
    "responses": {
        "200": {
            "description": "Successfully scraped content from the URL",
            "content": {
                "application/json": {
                    "schema": {
                        "type": "object",
                        "properties": {
                            "scraped_content": {
                                "type": "string",
                                "description": "The content scraped from the URL."
                            }
                        }
                    }
                }
            }
        },
        "400": {
            "description": "Bad request. The input URL is missing or invalid."
        }
    }
  }
 }
}
```



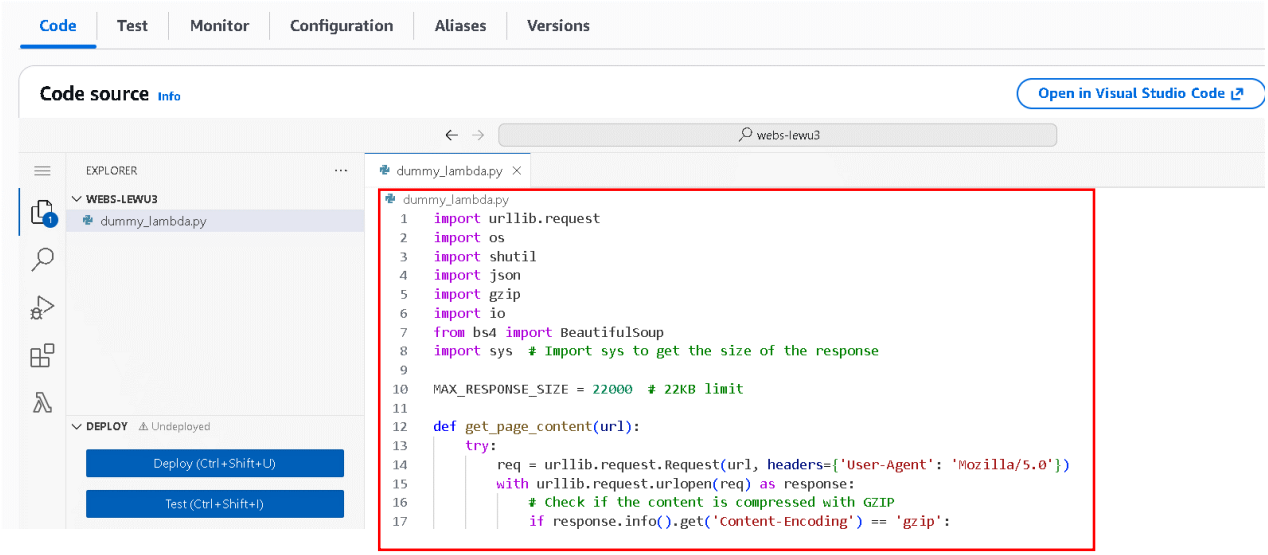
- Simply scroll down and click create.



- Now scroll down and see our webs action group is created.
- Let’s click on it and go inside.



- Inside scroll down and click to View to see created Lambda function



- Now we see already written Lambda function
- We change with our code.

```
import urllib.request
import os
import shutil
import json
import gzip
import io
from bs4 import BeautifulSoup
import sys  # Import sys to get the size of the response

MAX_RESPONSE_SIZE = 22000  # 22KB limit

def get_page_content(url):
    try:
        req = urllib.request.Request(url, headers={'User-Agent':
'Mozilla/5.0'})
        with urllib.request.urlopen(req) as response:
            # Check if the content is compressed with GZIP
            if response.info().get('Content-Encoding') == 'gzip':
                print(f"Content from {url} is GZIP encoded,
decompressing...")
                buf = io.BytesIO(response.read())
                with gzip.GzipFile(fileobj=buf) as f:
                    content = f.read().decode('utf-8')
            else:
                content = response.read().decode('utf-8')

            if response.geturl() != url:  # Check if there were any
redirects
                print(f"Redirect detected for {url}")
                return None

            return content
    except Exception as e:
        print(f"Error while fetching content from {url}: {e}")
        return None
```

```
def empty_tmp_folder():
    try:
        for filename in os.listdir('/tmp'):
            file_path = os.path.join('/tmp', filename)
            if os.path.isfile(file_path) or
os.path.islink(file_path):
                os.unlink(file_path)
            elif os.path.isdir(file_path):
                shutil.rmtree(file_path)
        print("Temporary folder emptied.")
        return "Temporary folder emptied."
    except Exception as e:
        print(f"Error while emptying /tmp folder: {e}")
        return None

def save_to_tmp(filename, content):
    try:
        if content is not None:
            print(content)
            with open(f'/tmp/{filename}', 'w') as file:
                file.write(content)
            print(f"Saved {filename} to /tmp")
            return f"Saved {filename} to /tmp"
        else:
            raise Exception("No content to save.")
    except Exception as e:
        print(f"Error while saving {filename} to /tmp: {e}")
        return None

def check_tmp_for_data(query):
    try:
        data = []
        for filename in os.listdir('/tmp'):
```

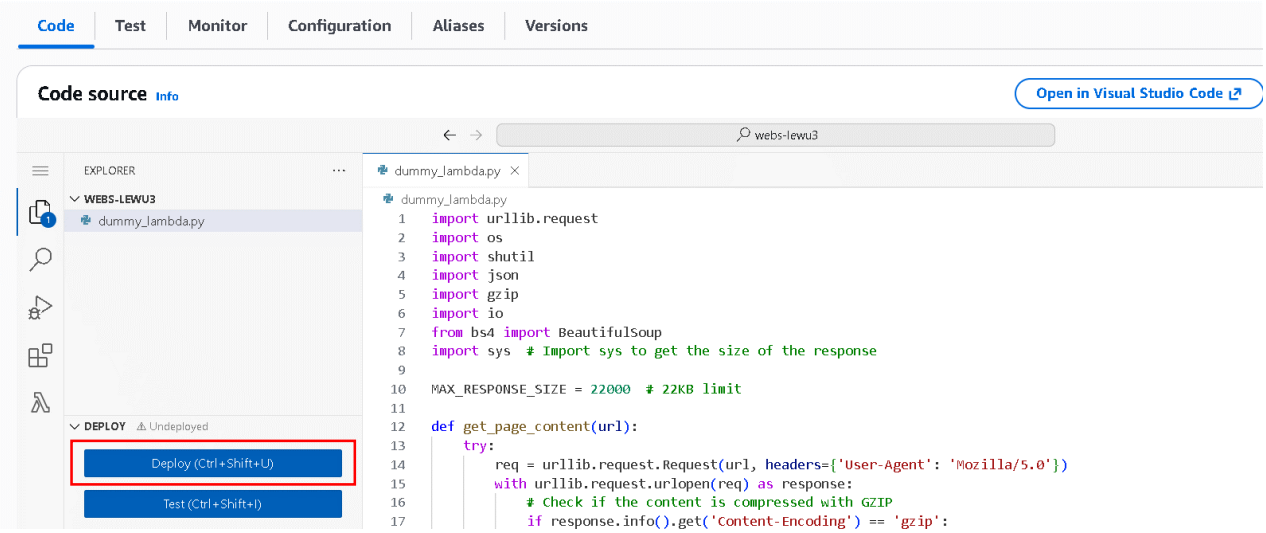
```
            if query in filename:
                with open(f'/tmp/{filename}', 'r') as file:
                    data.append(file.read())
        print(f"Found {len(data)} file(s) in /tmp for query {query}")
        return data if data else None
    except Exception as e:
        print(f"Error while checking /tmp for query {query}: {e}")
        return None


def handle_search(event):
    # Extract inputURL from the requestBody content
    request_body = event.get('requestBody', {})
    input_url = ''

    # Check if the inputURL exists within the properties
    if 'content' in request_body:
        properties = request_body['content'].get('application/json',
{}).get('properties', [])
        input_url = next((prop['value'] for prop in properties if
prop['name'] == 'inputURL'), '')

    # Handle missing URL
    if not input_url:
        return {"error": "No URL provided"}

    # Ensure URL starts with http or https
    if not input_url.startswith(('http://', 'https://')):
        input_url = 'http://' + input_url

    # Check for existing data in /tmp
    tmp_data = check_tmp_for_data(input_url)
    if tmp_data:
        return {"results": tmp_data}
```

```
    # Clear /tmp folder
    empty_tmp_result = empty_tmp_folder()
    if empty_tmp_result is None:
        return {"error": "Failed to empty /tmp folder"}

    # Get the page content
    content = get_page_content(input_url)
    if content is None:
        return {"error": "Failed to retrieve content"}

    # Parse and clean the HTML content
    cleaned_content = parse_html_content(content)

    # Save the content to /tmp
    filename = input_url.split('//')[-1].replace('/', '_') + '.txt'
    save_result = save_to_tmp(filename, cleaned_content)

    if save_result is None:
        return {"error": "Failed to save to /tmp"}

    # Check the size of the response and truncate if necessary
    response_data = {'url': input_url, 'content': cleaned_content}
    response_size = sys.getsizeof(json.dumps(response_data))

    if response_size > MAX_RESPONSE_SIZE:
        print(f"Response size {response_size} exceeds limit.
Truncating content...")
        truncated_content = cleaned_content[:(MAX_RESPONSE_SIZE -
response_size)]
        response_data['content'] = truncated_content

    return {"results": response_data}


def parse_html_content(html_content):
```

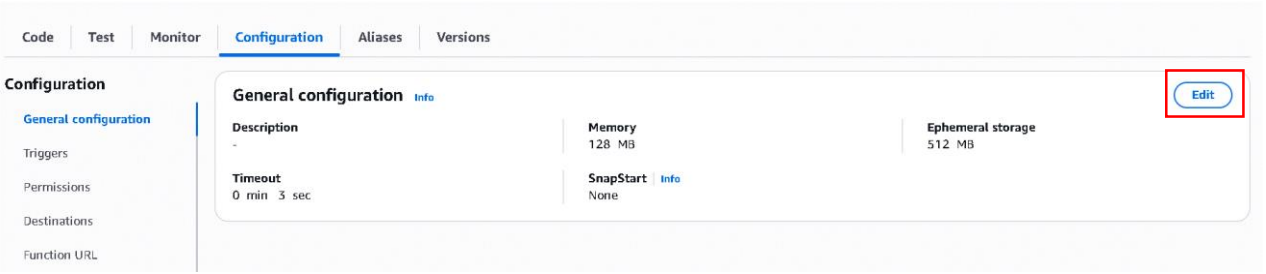
```
    soup = BeautifulSoup(html_content, 'html.parser')
    for script_or_style in soup(["script", "style"]):
        script_or_style.decompose()
    text = soup.get_text()
    lines = (line.strip() for line in text.splitlines())
    chunks = (phrase.strip() for line in lines for phrase in
line.split("  "))
    cleaned_text = '\n'.join(chunk for chunk in chunks if chunk)

    max_size = 25000
    if len(cleaned_text) > max_size:
        cleaned_text = cleaned_text[:max_size]

    return cleaned_text

def lambda_handler(event, context):
    response_code = 200
    action_group = event['actionGroup']
    api_path = event['apiPath']

    print("THE EVENT: ", event)

    if api_path == '/search':
        result = handle_search(event)
    else:
        response_code = 404
        result = f"Unrecognized api path: {action_group}::{api_path}"

    response_body = {
        'application/json': {
            'body': result
        }
    }
```

```
    action_response = {
        'actionGroup': event['actionGroup'],
        'apiPath': event['apiPath'],
        'httpMethod': event['httpMethod'],
        'httpStatusCode': response_code,
        'responseBody': response_body
    }

    api_response = {'messageVersion': '1.0', 'response':
action_response}
    print("action_response: ", action_response)
    print("response_body: ", response_body)
    return api_response
```

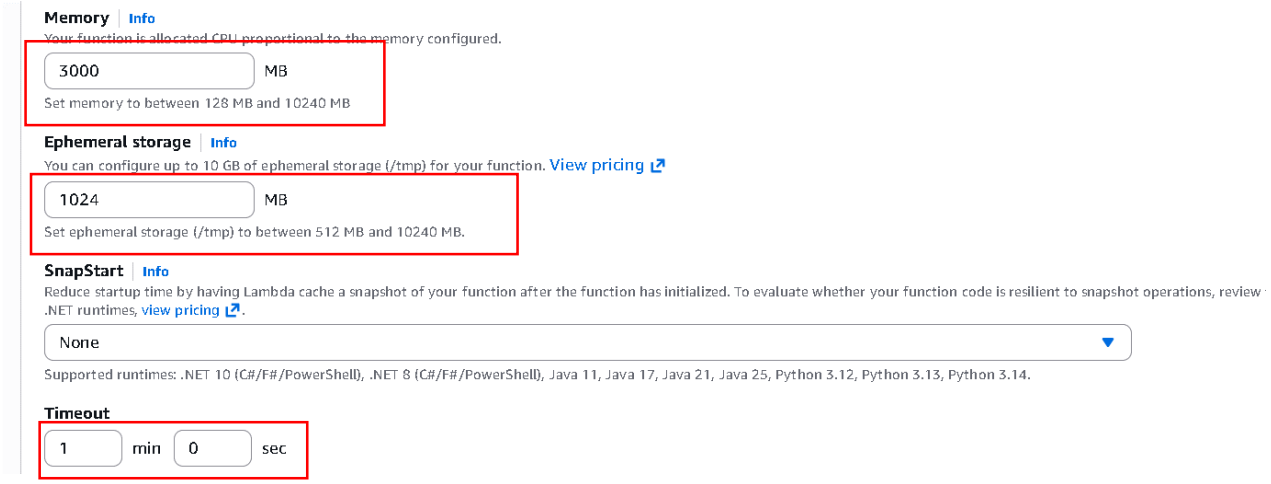
- Now this code gives a result and then this result is going to be then passed back to AWS or Amazon Bedrock Agent to then take that information and create a coherent response.



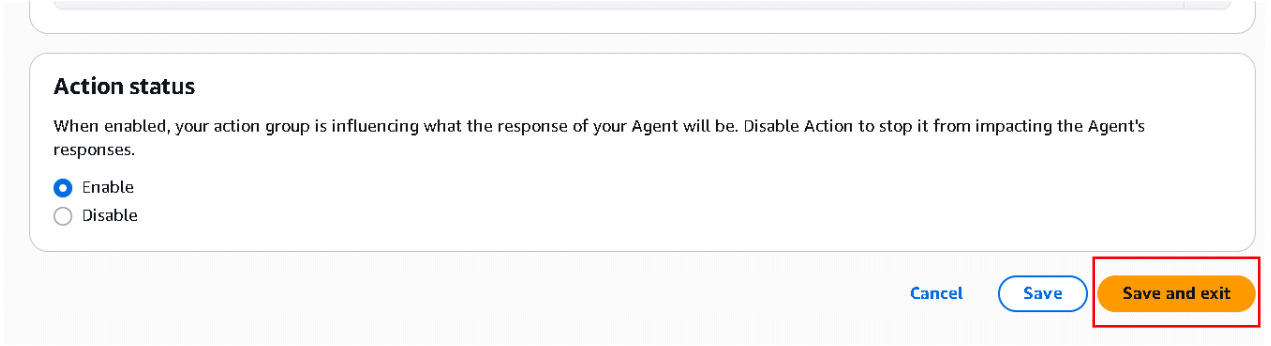
- Now simply click Deploy



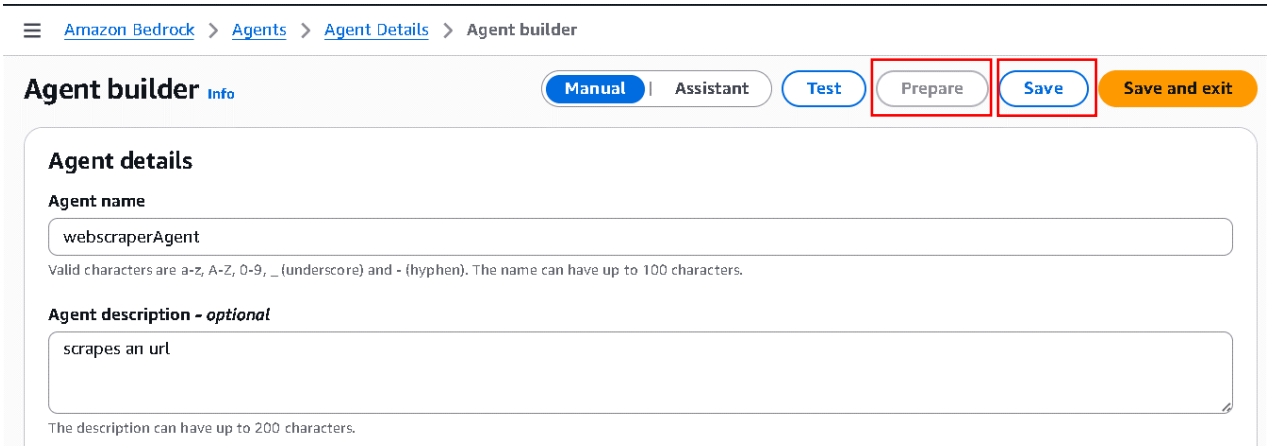
- Go to configuration and click Edit



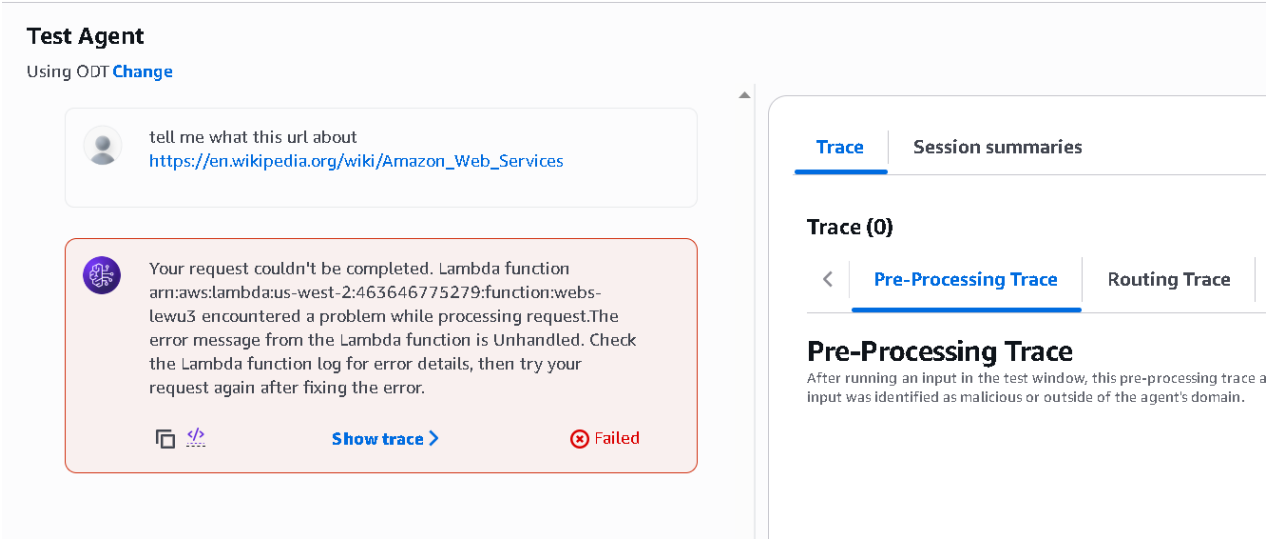
- Now just scroll down and click save.
- Now lambda function work is done go back to your Agent console.



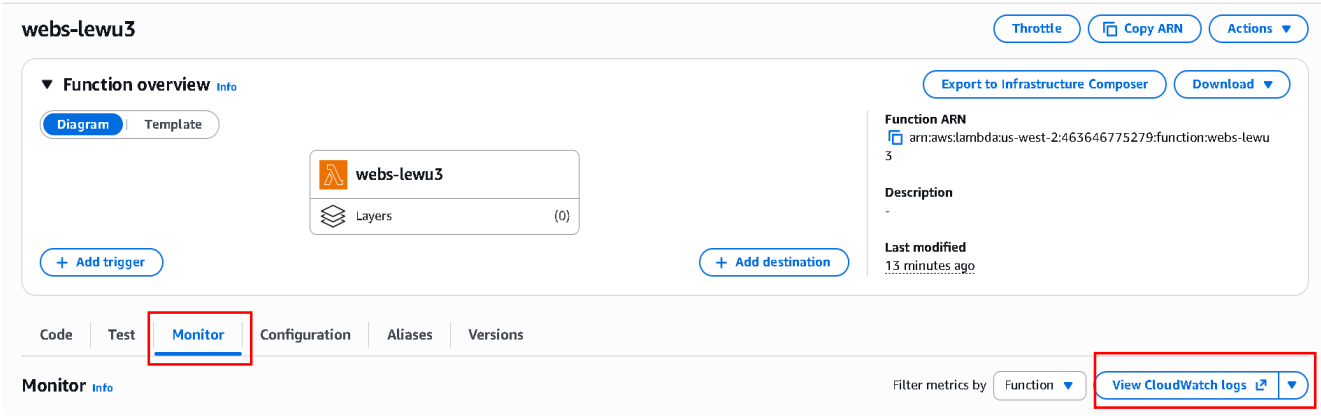
- Just scroll down and click save and exit.



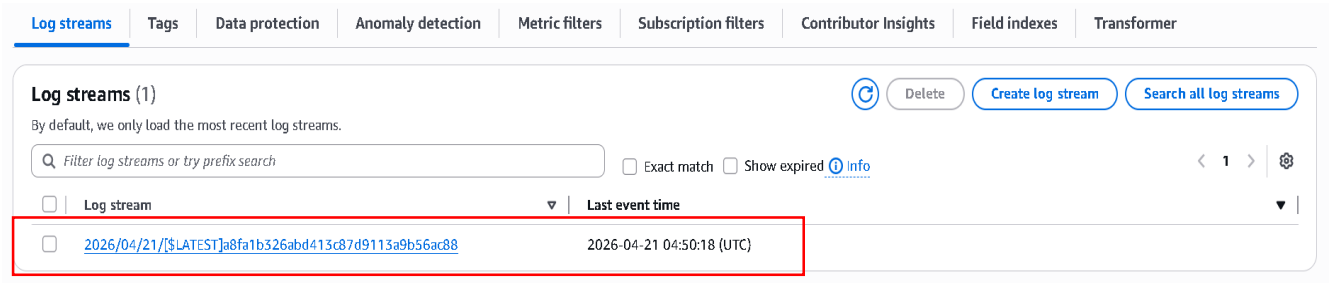
- Now just click save
- After save just click prepare.
- And then test your agent.



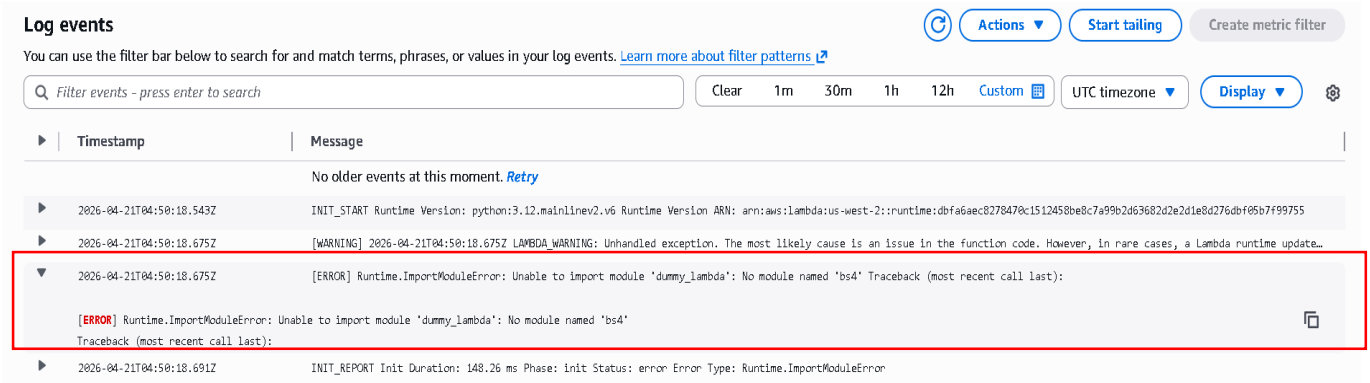
- You see I ask but request failed
- So let's resolve this
- Go back to tour lambda tab



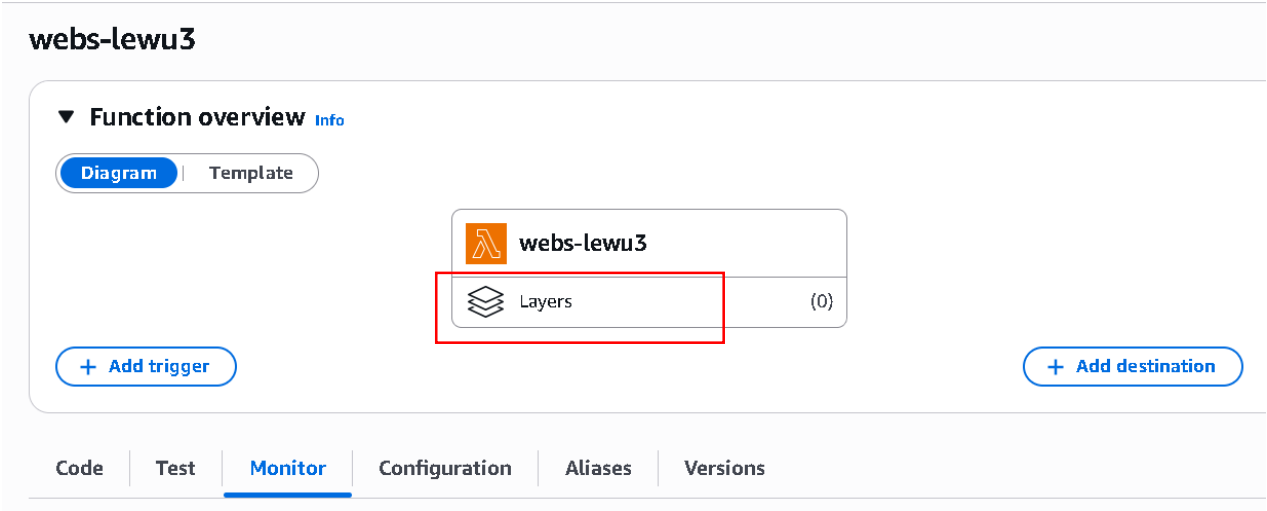
- Now in your lambda function click Monitor.
- Then in monitor click view cloudWatch logs.



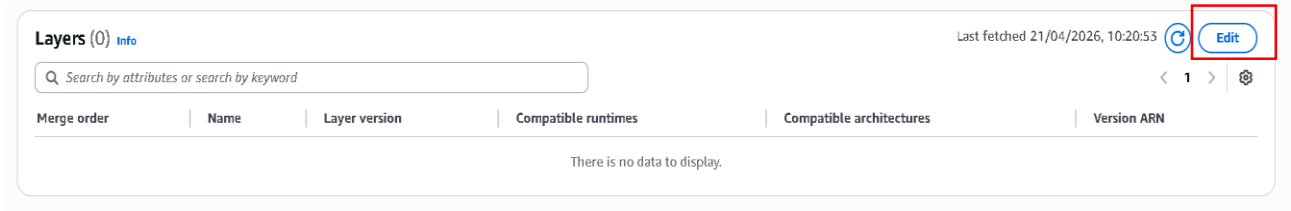
- Now you are redirected to new tab cloudWatch.
- Now just scroll down and in log stream click the log that you see



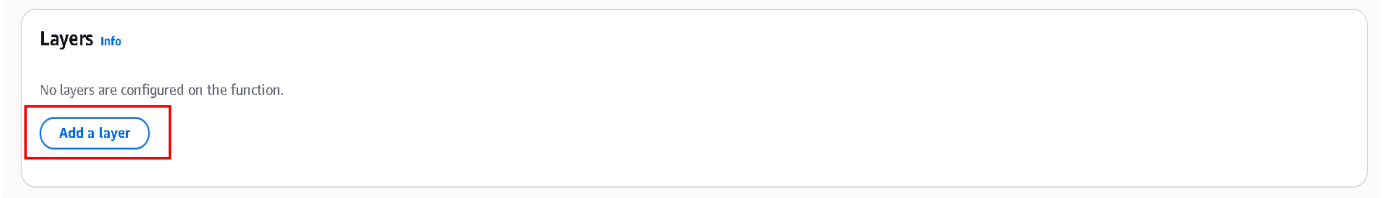
- Now you are in logs page.
- You see in third Line we get error.
- See that it says unable to import module ‘dummylambda’.
- Basically it means we are actually importing this beautiful soup IP and all of these other libraries.
- But what happened is that in this environment some of these libraries are not there.
- So which means they are not available.
- So what we can do we attach a layer in our lambda function.
- So go to your lambda function tab.



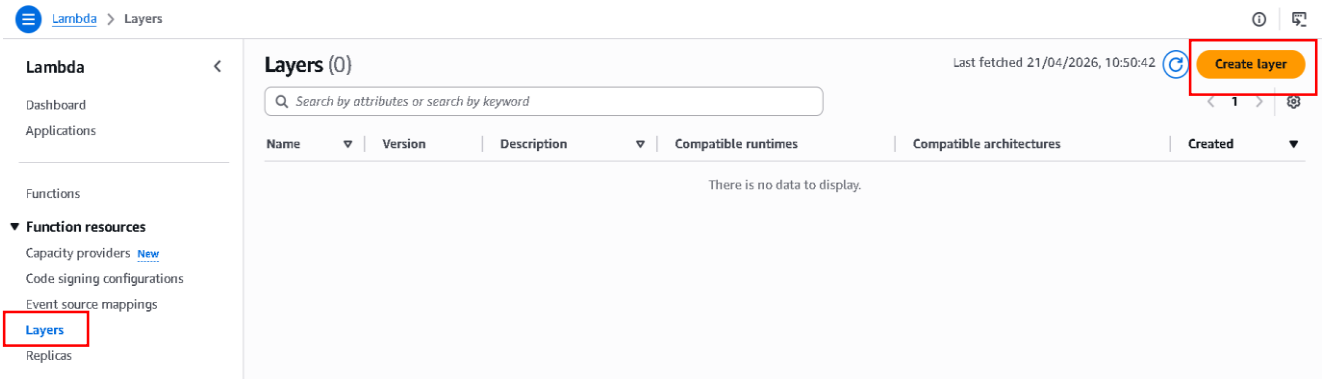
- Now click on Layers



- Now simply scroll down and in Layer Section.
- Click Edit.



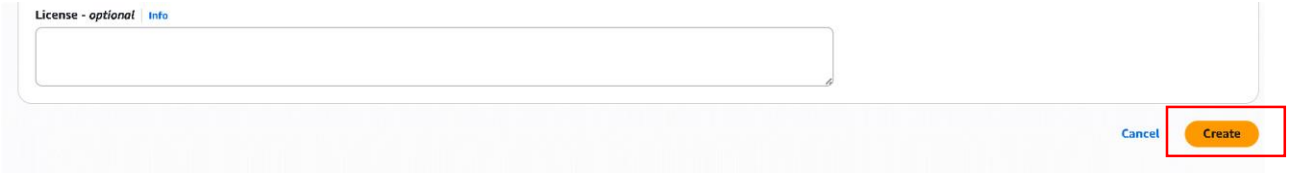
- Now you see Add a layer.
- But before you click you need to create a Layer
- So go to lambda function in a new tab.



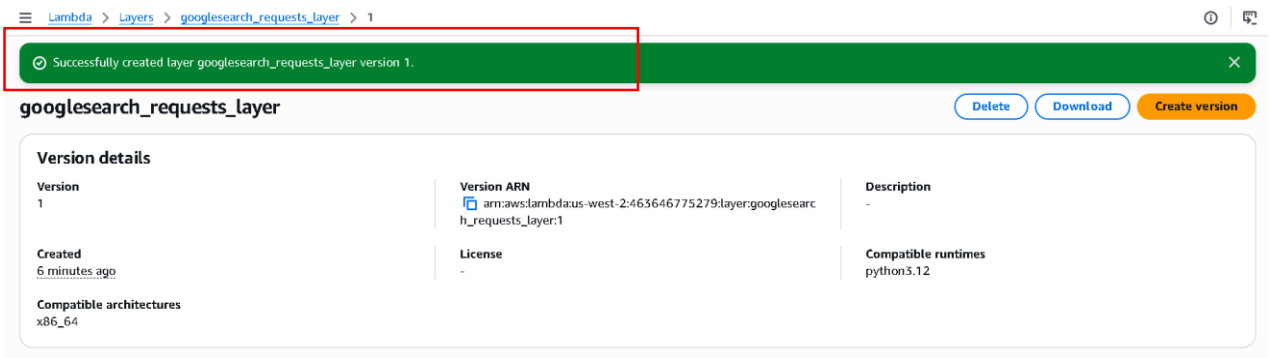
- Now just click Layer in Navigation bar under Functions section.
- And after this click create layer

- Give name to your layer.
- Select upload a .zip file.
- Click choose file and upload the file. (that file I provided in resource)
- Now just scroll down

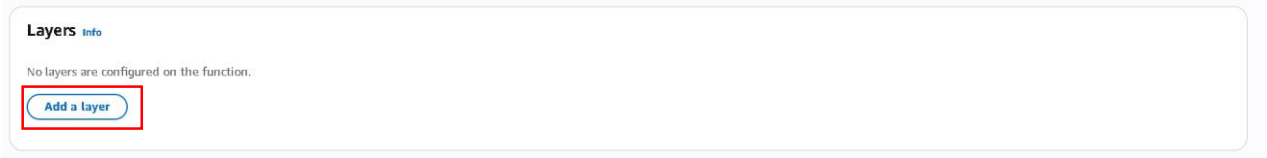
- Now select compatible architectures x86_64
- And then choose compatible runtimes Python 3.12
- Now simply scroll down.



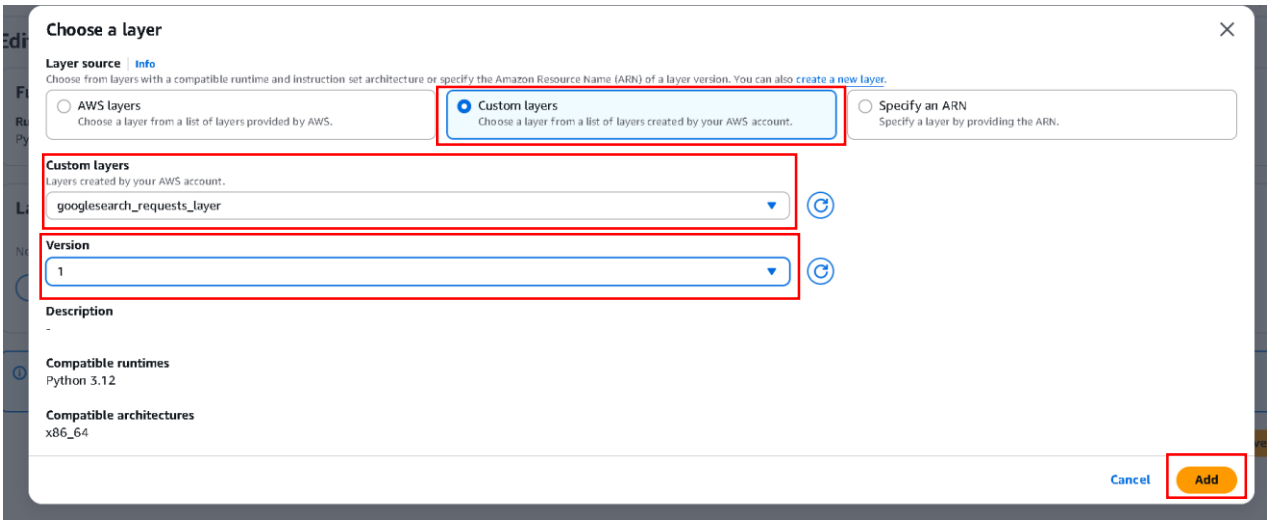
- Now just click Create.



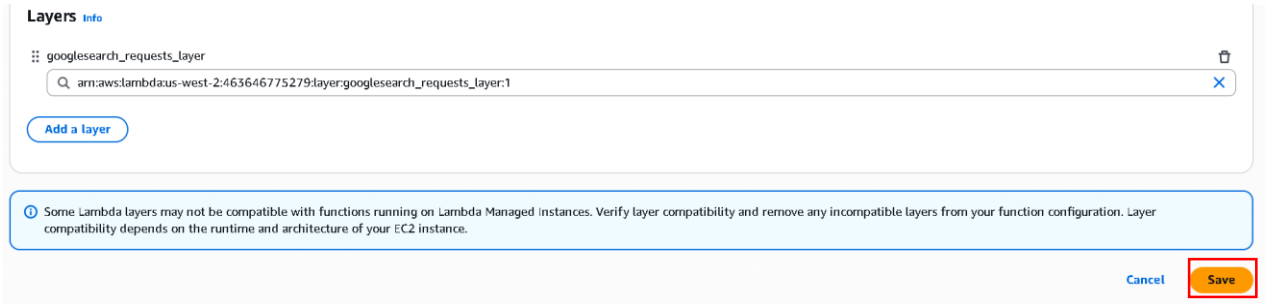
- Now our layer is successfully created.
- Now go back to your Lambda function tab.



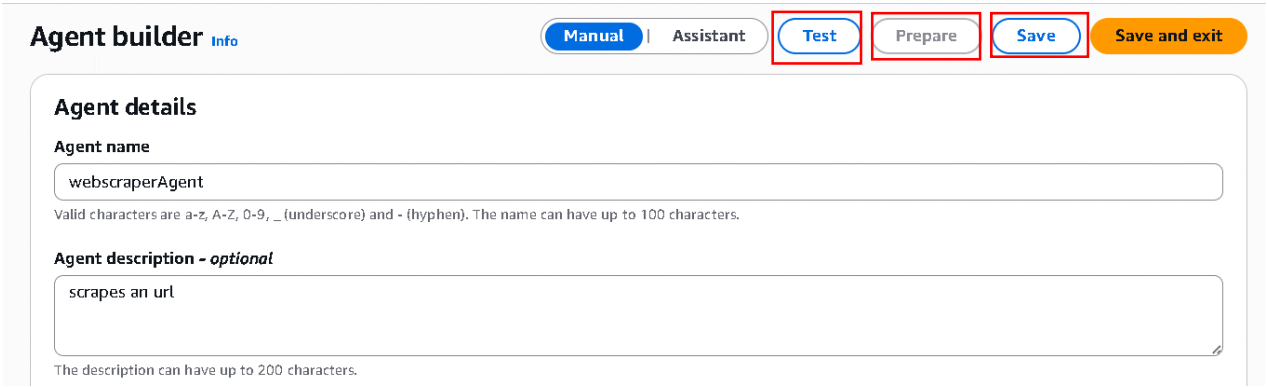
- Now just click Add a layer



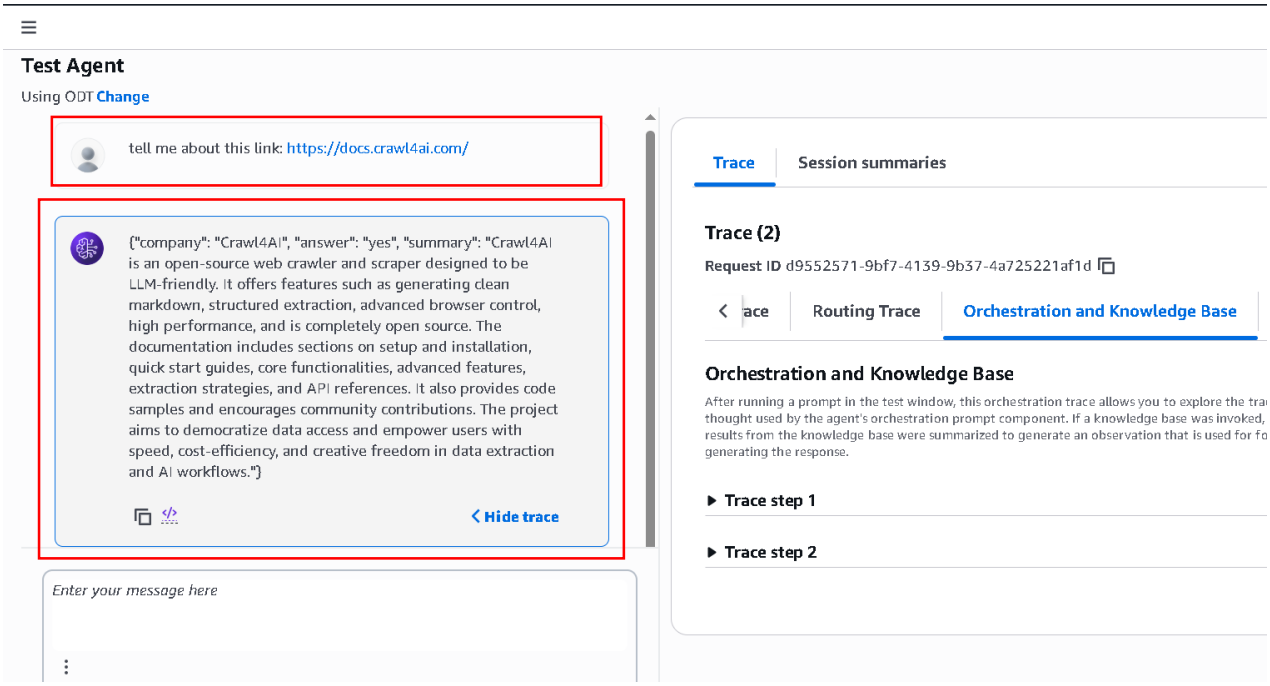
- choose custom Layer.
- Select your created Layers.
- And choose version as 1.
- And now just click Add.



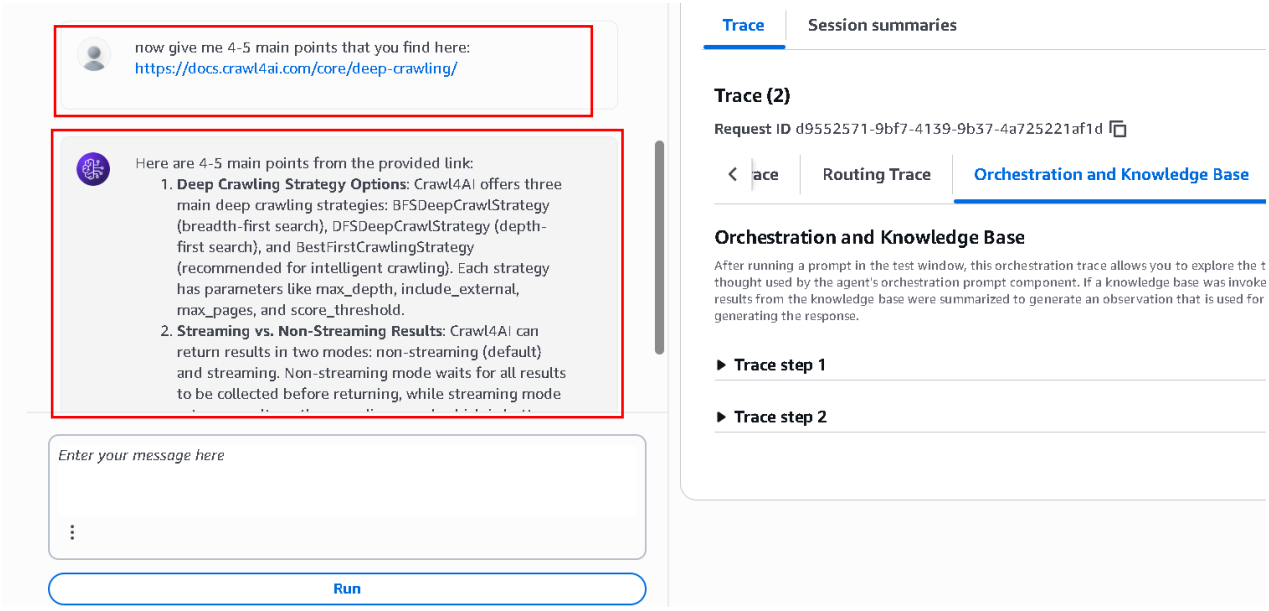
- Now just click Save.
- Go now go to your Amazon Bedrock Agent tab.



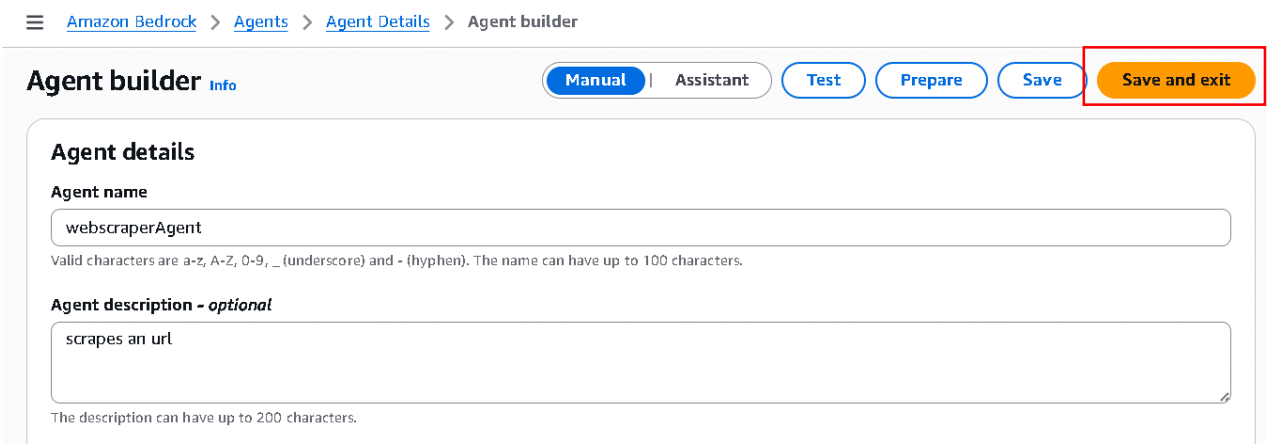
- Now click Save.
- After Save Prepare.
- After Prepare Test.



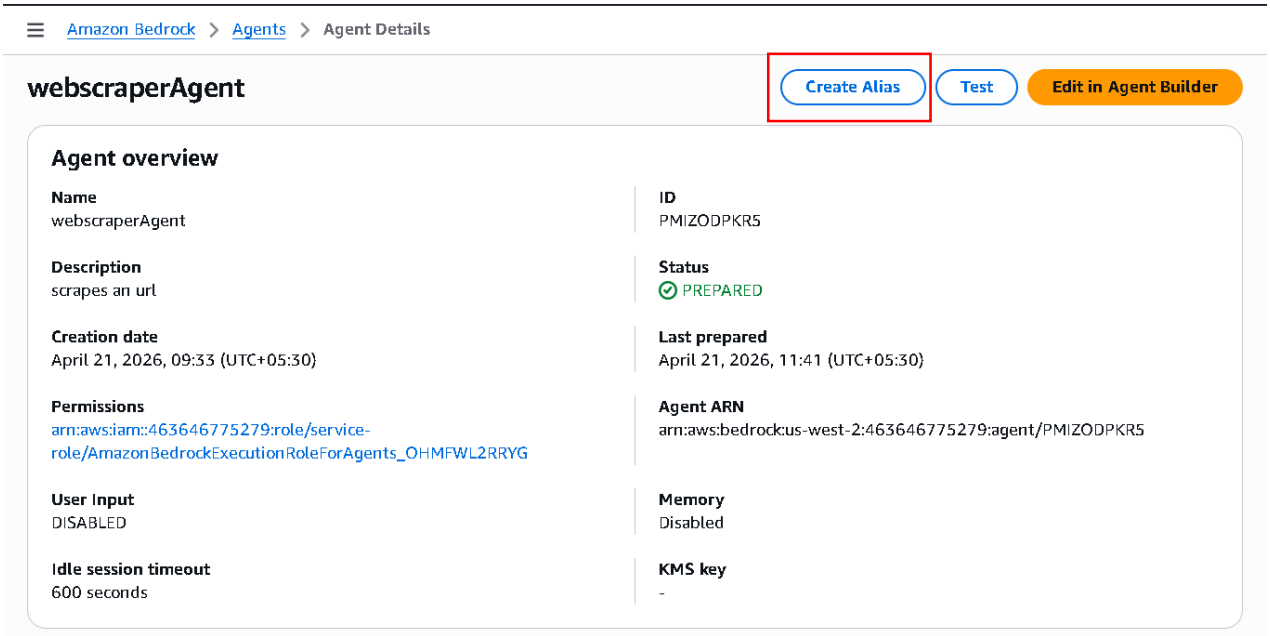
- I ask now it's working perfectly.
- Now ask some other Question.



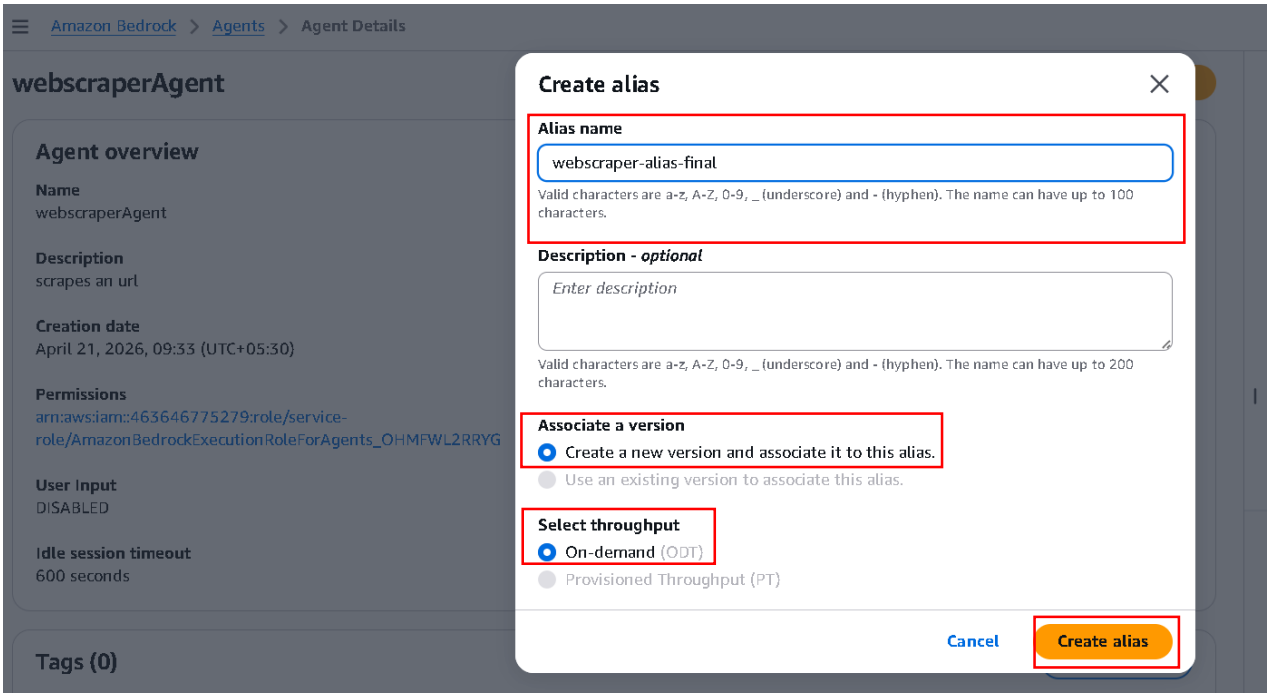
- It's working Perfectly.



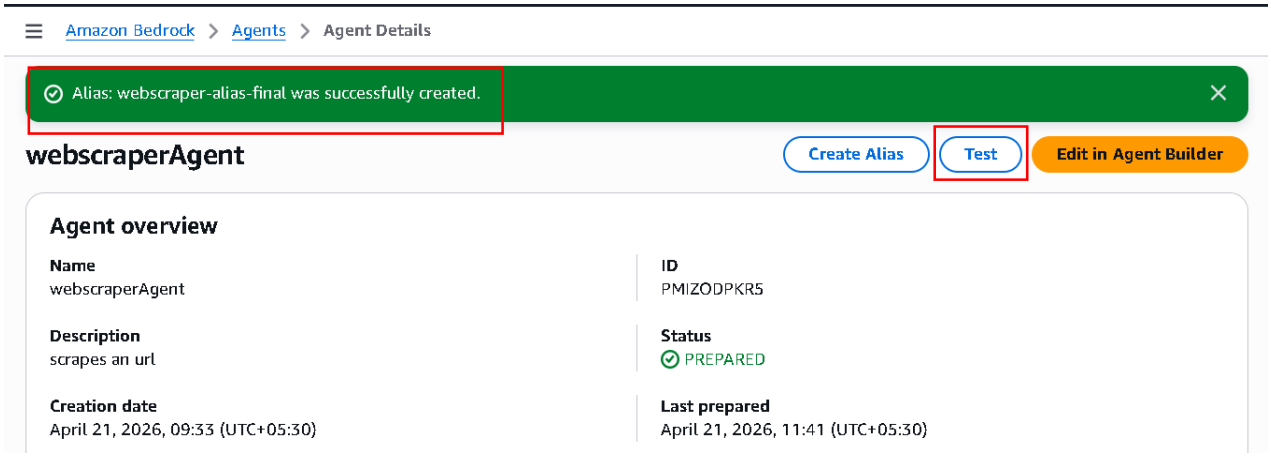
- For one more time click save, then Prepare.
- Now just click Save and Exit.



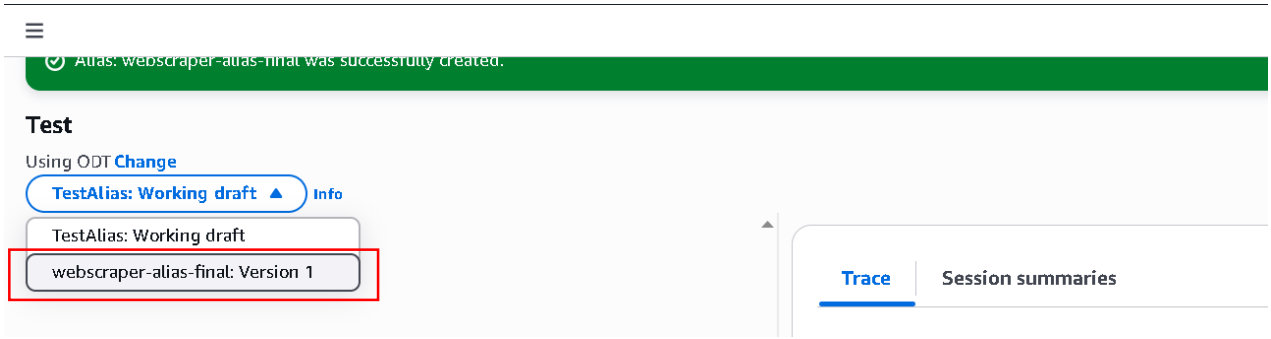
- Now simply click on create Alias.



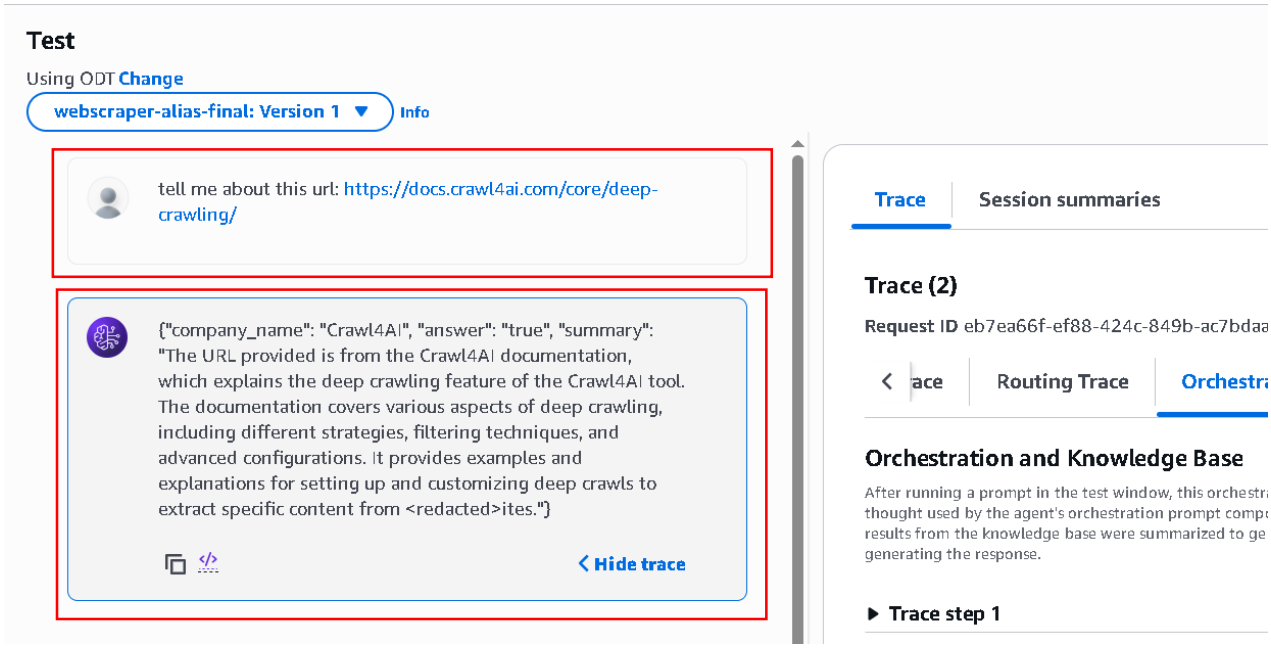
- Now simply write alias name.
- Associate a version
  - ➢ Create a new version and associate it to this alias.
- Select throughput
  - ➢ On-demand
- Now just simply click create alias.



- Our Alias is created now just click Test.



- Now select your created alias
- Now check it’s running or not



- It’s working Properly.